

Prática 5: Cálculo de π por Monte Carlo com MPI

Lucas Arruk Mendes¹
Pedro Augusto Faria²

¹ RA: 819446, Departamento de Computação, Universidade Federal de São Carlos

² RA: 821124, Departamento de Computação, Universidade Federal de São Carlos

Resumo:

Este tutorial apresenta a estimativa de π pelo método de Monte Carlo em Python, nas versões serial e paralela com quatro processos MPI. São descritos o método, a implementação, a execução e a comparação de desempenho. Para 10^7 pontos, foi obtido speedup de 3,96.

Palavras-chave: Monte Carlo; MPI; Python; paralelismo; speedup.

1 Introdução

O objetivo desta prática é calcular π pelo método de Monte Carlo e paralelizar o cálculo com a biblioteca MPI para Python. O enunciado solicita: deduzir o método; executar um número grande de pontos em quatro processos; comparar com a versão serial; apresentar o speedup; documentar equações, código e execução; e discutir a arquitetura paralela adequada.

A solução utiliza um único arquivo Python. Primeiro, o processo de identificador zero executa a versão serial. Em seguida, os pontos são divididos entre os processos MPI e os resultados locais são somados por uma operação coletiva.

2 Método de Monte Carlo

Considere um círculo de raio $r = 1$ inscrito em um quadrado de lado $2r$, como mostra a Figura 1. Suas áreas são

$$A_c = \pi r^2 = \pi \quad \text{e} \quad A_q = (2r)^2 = 4. \quad (1)$$

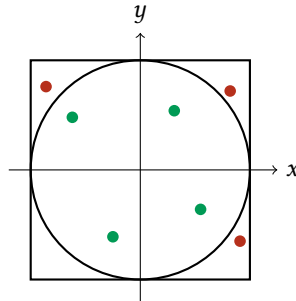


Figura 1. Círculo unitário inscrito no quadrado de lado 2.

São sorteados n pontos uniformes (x, y) no quadrado. Um ponto pertence ao círculo quando

$$x^2 + y^2 \leq 1. \quad (2)$$

Se d pontos estiverem dentro do círculo, a razão d/n aproxima a razão entre as áreas. Assim,

$$\frac{d}{n} \approx \frac{A_c}{A_q} = \frac{\pi}{4} \quad \Rightarrow \quad \pi \approx 4 \frac{d}{n}. \quad (3)$$

O erro é estatístico e tende a diminuir com o aumento de n , aproximadamente na ordem de $1/\sqrt{n}$.

3 Tutorial de implementação

3.1 Instalação

Em Debian ou Ubuntu, Open MPI e a interface Python podem ser instalados com:

```
sudo apt update
sudo apt install openmpi-bin python3-mpi4py
```

O programa usa `MPI.COMM_WORLD`, que reúne todos os processos iniciados pelo `mpiexec`. Cada processo consulta seu identificador (`rank`) e o número total de processos. A documentação oficial do `mpi4py` apresenta esse modelo de execução e as operações coletivas [MPI for Python, 2026](#).

3.2 Código

A função `calcular_pi` sorteia os pontos e retorna apenas a quantidade que pertence ao círculo. Na parte paralela, cada processo recebe aproximadamente N/p pontos. A operação `reduce`, com `MPI.SUM`, soma os acertos e entrega o total ao processo zero.

Listing 1: Implementação serial e MPI em um único arquivo.

```

1 from mpi4py import MPI
2 from random import random, seed
3 from time import perf_counter
4
5 N = 10_000_000
6
7
8 def calcular_pi(n):
9     # Conta os pontos sorteados dentro do círculo unitário.
10    dentro = sum(random() ** 2 + random() ** 2 <= 1 for _ in range(n))
11    return dentro
12
13
14 comm = MPI.COMM_WORLD
15 rank = comm.Get_rank()
16 processos = comm.Get_size()
17
18 # 0 processo zero mede primeiro a implementação serial.
19 if rank == 0:
20     seed(0)
21     inicio = perf_counter()
22     dentro_serial = calcular_pi(N)
23     tempo_serial = perf_counter() - inicio
24     pi_serial = 4 * dentro_serial / N
25
26 # Cada processo calcula uma parcela dos pontos.
27 quantidade = N // processos + (rank < N % processos)
28 seed(rank + 1)
29 comm.Barrier()
30 inicio = MPI.Wtime()
31 dentro_local = calcular_pi(quantidade)
32 dentro_total = comm.reduce(dentro_local, op=MPI.SUM, root=0)
33 tempo_paralelo = MPI.Wtime() - inicio
34
35 # Apenas o processo zero recebe o total e apresenta os resultados.
36 if rank == 0:
37     pi_paralelo = 4 * dentro_total / N
38     print(f"Pi serial: {pi_serial}")
39     print(f"Tempo serial: {tempo_serial:.4f} s")
40     print(f"Pi paralelo: {pi_paralelo}")
41     print(f"Tempo MPI: {tempo_paralelo:.4f} s")
42     print(f"Speedup: {tempo_serial / tempo_paralelo:.2f}")

```

3.3 Execução

O programa é executado com quatro processos, como solicitado:

```
mpiexec -n 4 python3 pi.py
```

O `mpiexec` é o comando padrão para iniciar os processos MPI [Open MPI, 2026](#). Caso o Open MPI emita avisos sobre interfaces TCP em uma execução inteiramente local, pode-se selecionar apenas os transportes locais:

```
mpiexec --mca btl self,sm -n 4 python3 pi.py
```

4 Resultados

O experimento utilizou $N = 10\,000\,000$ pontos e quatro processos na mesma máquina multicore. A Tabela 1 resume a execução.

Tabela 1 — Resultados das implementações serial e MPI.

Implementação	π estimado	Tempo (s)
Serial	3,1413028	1,7965
MPI, 4 processos	3,1414560	0,4533

O speedup é calculado por

$$S_p = \frac{T_s}{T_p} = \frac{1,7965}{0,4533} \approx 3,96. \quad (4)$$

Para quatro processos, a eficiência foi

$$E_p = \frac{S_p}{p} = \frac{3,96}{4} \approx 0,99, \quad (5)$$

ou aproximadamente 99%. Os valores estimados diferem porque as versões usam sequências pseudoaleatórias distintas, mas ambas se aproximam de π .

5 Discussão

O problema é altamente paralelizável: cada ponto pode ser gerado e testado sem depender dos demais. A comunicação ocorre apenas no final, quando os acertos locais são somados. Essa baixa comunicação explica o speedup próximo do ideal observado no teste.

Do ponto de vista de Flynn, a solução é MIMD, pois vários processos executam instruções independentes sobre dados diferentes. O estilo de programação é SPMD: todos executam o mesmo programa, mas cada *rank* trabalha sobre uma parcela dos pontos.

MPI adota o modelo de memória distribuída entre processos. Na máquina multicore usada, a memória física é compartilhada, porém cada processo possui seu próprio espaço de endereçamento. Essa abordagem também permite executar o mesmo código em várias máquinas. Para este problema, MPI é adequado porque o trabalho pode ser dividido com pouca troca de mensagens.

O speedup pode diminuir para poucos pontos, pois inicialização, sincronização e redução passam a representar uma fração maior do tempo total. Também não se espera que o ganho cresça indefinidamente devido à parte serial e aos recursos limitados da máquina.

6 Conclusão

O método de Monte Carlo foi implementado em Python nas versões serial e MPI, em um único arquivo. Com 10^7 pontos e quatro processos, o tempo caiu de 1,7965 s para 0,4533 s, resultando em speedup de 3,96 e eficiência próxima de 99%. O resultado confirma que o cálculo é apropriado para paralelismo por processos, devido à independência entre as amostras e à pequena necessidade de comunicação.

Referências

MPI for Python (2026). *Tutorial*. URL: <https://mpi4py.readthedocs.io/en/stable/tutorial.html> (acesso em 10/06/2026).

Open MPI (2026). *mpirun / mpiexec manual*. URL: <https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man1/mpirun.1.html> (acesso em 10/06/2026).